

JOB SHEET 14

Tree

NAMA : M. Javier Thufail

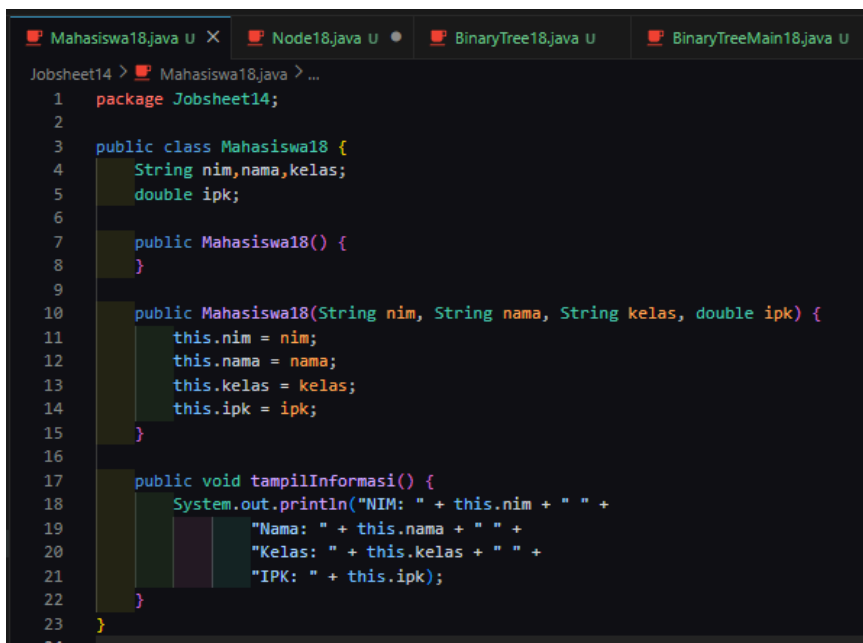
NIM : 254107020019

KELAS : TI 1-G

NO. ABSEN : 18

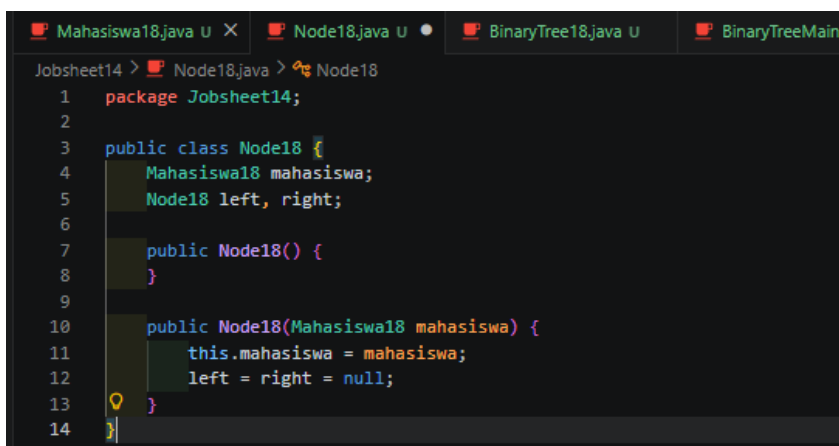
Praktikum 1 (Implementasi Binary Search Tree menggunakan Linked List).

Code Mahasiswa18.java:



```
1 package Jobsheet14;
2
3 public class Mahasiswa18 {
4     String nim,nama,kelas;
5     double ipk;
6
7     public Mahasiswa18() {
8     }
9
10    public Mahasiswa18(String nim, String nama, String kelas, double ipk) {
11        this.nim = nim;
12        this.nama = nama;
13        this.kelas = kelas;
14        this.ipk = ipk;
15    }
16
17    public void tampilInformasi() {
18        System.out.println("NIM: " + this.nim + " " +
19        "Nama: " + this.nama + " " +
20        "Kelas: " + this.kelas + " " +
21        "IPK: " + this.ipk);
22    }
23 }
24
```

Code Node18.java:



```
1 package Jobsheet14;
2
3 public class Node18 {
4     Mahasiswa18 mahasiswa;
5     Node18 left, right;
6
7     public Node18() {
8     }
9
10    public Node18(Mahasiswa18 mahasiswa) {
11        this.mahasiswa = mahasiswa;
12        left = right = null;
13    }
14 }
```

Code BinaryTree18.java:

```
JobSheet14 > BinaryTree18.java > ...
1 package Jobsheet14;
2
3 public class BinaryTree18 {
4     Node18 root;
5
6     public BinaryTree18() {
7         root = null;
8     }
9
10    public boolean isEmpty() {
11        return root == null;
12    }
13
14    public void add(Mahasiswa18 mahasiswa) {
15        Node18 newNode = new Node18(mahasiswa);
16        if (isEmpty()) {
17            root = newNode;
18        } else {
19            Node18 current = root;
20            Node18 parent = null;
21            while (true) {
22                parent = current;
23                if (mahasiswa.ipk < current.mahasiswa.ipk) {
24                    current = current.left;
25                    if (current == null) {
26                        parent.left = newNode;
27                        return;
28                    }
29                } else {
30                    current = current.right;
31                    if (current == null) {
32                        parent.right = newNode;
33                        return;
34                    }
35                }
36            }
37        }
38    }
39
40    boolean find(double ipk) {
41        boolean result = false;
42        Node18 current = root;
43        while (current != null) {
44            if (current.mahasiswa.ipk == ipk) {
45                result = true;
46                break;
47            } else if (ipk > current.mahasiswa.ipk) {
48                current = current.right;
49            } else {
50                current = current.left;
51            }
52        }
53        return result;
54    }
55
56    void traversePreOrder(Node18 node) {
57        if (node != null) {
58            node.mahasiswa.tampilInformasi();
59            traversePreOrder(node.left);
60            traversePreOrder(node.right);
61        }
62    }
63
64    void traverseInOrder(Node18 node) {
65        if (node != null) {
66            traverseInOrder(node.left);
67            node.mahasiswa.tampilInformasi();
68            traverseInOrder(node.right);
69        }
70    }
71}
```

```
Mahasiswa18.java > Node18.java > BinaryTree18.java > BinaryTreeMain18.java > BinaryTreeMainArray18.java > ...
JobSheet14 > BinaryTree18.java > ...
1 public class BinaryTree18 {
2
3     void traversePostOrder(Node18 node) {
4         if (node != null) {
5             traversePostOrder(node.left);
6             traversePostOrder(node.right);
7             node.mahasiswa.tampilInformasi();
8         }
9     }
10
11    Node18 getSuccessor(Node18 del) {
12        Node18 successor = del.right;
13        Node18 successorParent = del;
14        while (successor.left != null) {
15            successorParent = successor;
16            successor = successor.left;
17        }
18        if (successor != del.right) {
19            successorParent.left = successor.right;
20            successor.right = del.right;
21        }
22        return successor;
23    }
24
25    void deleteNode(double ipk) {
26        if (isEmpty()) {
27            System.out.println("Binary tree kosong");
28            return;
29        }
30        Node18 parent = root;
31        Node18 current = root;
32        boolean isLeftChild = false;
33        while (current != null) {
34            if (current.mahasiswa.ipk == ipk) {
35                return;
36            } else if (ipk < current.mahasiswa.ipk) {
37                parent = current;
38                current = current.left;
39                isLeftChild = true;
40            } else if (ipk > current.mahasiswa.ipk) {
41                parent = current;
42                current = current.right;
43                isLeftChild = false;
44            }
45        }
46        // penghapusan
47        if (current == null) {
48            System.out.println("Data tidak ditemukan");
49            return;
50        } else {
51            if (current.left == null && current.right == null) {
52                if (current == root) {
53                    root = null;
54                } else {
55                    if (isLeftChild) {
56                        parent.left = null;
57                    } else {
58                        parent.right = null;
59                    }
60                }
61            } else if (current.left == null) {
62                if (current == root) {
63                    root = current.right;
64                } else {
65                    if (isLeftChild) {
66                        parent.left = current.right;
67                    } else {
68                        parent.right = current.right;
69                    }
70                }
71            } else if (current.right == null) {
72                if (current == root) {
73                    root = current.left;
74                } else {
75                    if (isLeftChild) {
76                        parent.left = current.left;
77                    } else {
78                        parent.right = current.left;
79                    }
80                }
81            } else {
82                Node18 successor = getSuccessor(current);
83                System.out.println("Jika 2 anak, current = ");
84                successor.mahasiswa.tampilInformasi();
85                if (current == root) {
86                    root = successor;
87                } else {
88                    if (isLeftChild) {
89                        parent.left = successor;
90                    } else {
91                        parent.right = successor;
92                    }
93                }
94                successor.left = current.left;
95            }
96        }
97    }
98}
```

Code BinaryTreeMain18.java:

```
Jobsheet14 > BinaryTreeMain18.java > BinaryTreeMain18
1 package Jobsheet14;
2
3 public class BinaryTreeMain18 {
4     public static void main(String[] args) {
5         BinaryTree18 bst = new BinaryTree18();
6
7         bst.add(new Mahasiswa18(nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57));
8         bst.add(new Mahasiswa18(nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.85));
9         bst.add(new Mahasiswa18(nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.21));
10        bst.add(new Mahasiswa18(nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.54));
11
12        System.out.println(x: "\nDaftar semua mahasiswa (in order traversal):");
13        bst.traverseInOrder(bst.root);
14
15        System.out.println(x: "\nPencarian data mahasiswa:");
16        System.out.print(s: "Cari mahasiswa dengan ipk: 3.54 ");
17        String hasilCari = bst.find(ipk: 3.54) ? "Ditemukan" : "Tidak ditemukan";
18        System.out.println(hasilCari);
19
20        System.out.print(s: "Cari mahasiswa dengan ipk: 3.22: ");
21        hasilCari = bst.find(ipk: 3.22) ? "Ditemukan" : "Tidak ditemukan";
22        System.out.println(hasilCari);
23
24        bst.add(new Mahasiswa18(nim: "244160131", nama: "Devi", kelas: "A", ipk: 3.72));
25        bst.add(new Mahasiswa18(nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.37));
26        bst.add(new Mahasiswa18(nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.46));
27
28        System.out.println(x: "\nDaftar semua mahasiswa setelah penambahan 3 mahasiswa:");
29        System.out.println(x: "InOrder Traversal:");
30        bst.traverseInOrder(bst.root);
31
32        System.out.println(x: "\nPreOrder Traversal:");
33        bst.traversePreOrder(bst.root);
34
35        System.out.println(x: "\nPostOrder Traversal:");
36        bst.traversePostOrder(bst.root);
37
38        System.out.println(x: "\nPenghapusan data mahasiswa");
39        bst.delete(ipk: 3.57);
40
41        System.out.println(x: "\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal) ");
42        bst.traverseInOrder(bst.root);
43    }
44 }
```

Outputnya:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS
PS C:\ITI\ASD\PraktikumAlgoritmaStrukturData> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\user\AppData\Roaming\Code\User\workspaceStorage\536Fd7849
ws\PraktikumAlgoritmaStrukturData_b46f464f\bin' 'Jobsheet14.BinaryTreeMain18'

Daftar semua mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 Ditemukan
Cari mahasiswa dengan ipk: 3.22: Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal)
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
PS C:\ITI\ASD\PraktikumAlgoritmaStrukturData>
```

Pertanyaan!

1. Karena pada Binary Search Tree (BST), data tersusun secara terurut. Nilai di kiri selalu lebih kecil, sedangkan nilai di kanan lebih besar. Dengan aturan ini, proses pencarian jadi lebih cepat dibanding binary tree biasa karena tidak perlu memeriksa semua node.
2. Atribut left dan right digunakan untuk menyimpan referensi anak node.
 - left menunjuk ke anak kiri dengan nilai lebih kecil.
 - right menunjuk ke anak kanan dengan nilai lebih besar atau sama.
3.
 - a. Atribut root digunakan untuk menyimpan node paling atas atau akar dari tree.
 - b. Saat objek tree pertama kali dibuat, nilai root adalah null.
4. Jika tree masih kosong, node baru yang ditambahkan akan langsung menjadi root.
5. Kode tersebut digunakan untuk menentukan posisi node baru berdasarkan nilai IPK.
 - Jika IPK lebih kecil dari node saat ini, pencarian diarahkan ke kiri.
 - Jika posisi kiri kosong, node baru ditempatkan di kiri parent.
 - Jika IPK lebih besar atau sama, pencarian diarahkan ke kanan.
 - Jika posisi kanan kosong, node baru ditempatkan di kanan parent.
6. Saat menghapus node yang memiliki dua anak, method delete() akan mencari node pengganti menggunakan getSuccessor(), yaitu node dengan nilai terkecil di subtree kanan. Setelah itu, posisi node yang dihapus diganti oleh successor, lalu hubungan anak kiri dan kanan disambungkan kembali agar struktur tree tetap benar.

Praktikum 2 (Implementasi Binary Tree dengan Array).

Code BinaryTreeArray18.java:

```
Mahasiswa18.java BinaryTreeArray18.java U BinaryTreeMainArray18.java U
Jobsheet14 > BinaryTreeArray18.java > ...
1 package Jobsheet14;
2
3 public class BinaryTreeArray18 {
4     Mahasiswa18[] dataMahasiswa;
5     int idxLast;
6
7     public BinaryTreeArray18() {
8         this.dataMahasiswa = new Mahasiswa18[10];
9     }
10
11     void populateData(Mahasiswa18 dataMhs[], int idxLast) {
12         this.dataMahasiswa = dataMhs;
13         this.idxLast = idxLast;
14     }
15
16     void traverseInOrder(int idxStart) {
17         if (idxStart <= idxLast) {
18             if (dataMahasiswa[idxStart] != null) {
19                 traverseInOrder(2 * idxStart + 1);
20                 dataMahasiswa[idxStart].tampilInformasi();
21                 traverseInOrder(2 * idxStart + 2);
22             }
23         }
24     }
25 }
26
```

Code BinaryTreeMainArray18.java dan Outputnya:

```
Mahasiswa18.java BinaryTreeArray18.java U BinaryTreeMainArray18.java U
Jobsheet14 > BinaryTreeMainArray18.java > BinaryTreeMainArray18
1 package Jobsheet14;
2
3 public class BinaryTreeMainArray18 {
4     public static void main(String[] args) {
5         BinaryTreeArray18 bta = new BinaryTreeArray18();
6         Mahasiswa18 mhs1 = new Mahasiswa18(nim: "244160121", nama: "Ali", kelas: "A", ipk: 3.57);
7         Mahasiswa18 mhs2 = new Mahasiswa18(nim: "244160185", nama: "Candra", kelas: "C", ipk: 3.41);
8         Mahasiswa18 mhs3 = new Mahasiswa18(nim: "244160221", nama: "Badar", kelas: "B", ipk: 3.75);
9         Mahasiswa18 mhs4 = new Mahasiswa18(nim: "244160220", nama: "Dewi", kelas: "B", ipk: 3.35);
10
11         Mahasiswa18 mhs5 = new Mahasiswa18(nim: "244160131", nama: "Devi", kelas: "A", ipk: 3.48);
12         Mahasiswa18 mhs6 = new Mahasiswa18(nim: "244160205", nama: "Ehsan", kelas: "D", ipk: 3.61);
13         Mahasiswa18 mhs7 = new Mahasiswa18(nim: "244160170", nama: "Fizi", kelas: "B", ipk: 3.86);
14
15         Mahasiswa18[] dataMahasiswas = { mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7, null, null, null };
16         int idxLast = 6;
17         bta.populateData(dataMahasiswas, idxLast);
18         System.out.println(x: "\nInorder Traversal Mahasiswa: ");
19         bta.traverseInOrder(idxStart: 0);
20     }
21 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetails -ea User\workspaceStorage\536fd784924cf6d1cf999570c8a955a3\redhat.java\jdt_ws\PraktikumAlgoritmaStrukturData_b46f4

Inorder Traversal Mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>
```

Pertanyaan!

1. Atribut data digunakan untuk menyimpan node-node binary tree dalam bentuk array. Sedangkan idxLast berfungsi menyimpan indeks terakhir yang berisi data valid agar proses traversal tidak melewati batas array.
2. Method populateData() digunakan untuk mengisi atribut tree dengan data yang dimasukkan dari luar.
3. Method traverseInOrder() digunakan untuk menampilkan isi binary tree dengan urutan In-Order, yaitu kiri → root → kanan.
4. Jika suatu node berada pada indeks 2, maka:
 - left child berada di indeks $(2 \times 2) + 1 = 5$
 - right child berada di indeks $(2 \times 2) + 2 = 6$
5. Statement `int idxLast = 6` digunakan sebagai penanda bahwa indeks terakhir yang berisi data valid adalah indeks ke-6.
6. Indeks $2 * idxStart + 1$ dan $2 * idxStart + 2$ digunakan untuk mencari posisi anak kiri dan anak kanan dalam array. Hal ini karena binary tree disimpan secara linear di dalam array, sehingga hubungan parent dan child ditentukan melalui perhitungan indeks, bukan menggunakan pointer seperti pada linked structure.

Tugas Praktikum.

1.

```
57 // 1. method addRekursif
58 public void addRekursif(Mahasiswa18 mahasiswa) {
59     root = addRekursif(root, mahasiswa);
60 }
61
62 private Node18 addRekursif(Node18 current, Mahasiswa18 mahasiswa) {
63     if (current == null) {
64         return new Node18(mahasiswa);
65     }
66
67     if (mahasiswa.ipk < current.mahasiswa.ipk) {
68         current.left = addRekursif(current.left, mahasiswa);
69     } else if (mahasiswa.ipk > current.mahasiswa.ipk) {
70         current.right = addRekursif(current.right, mahasiswa);
71     }
72
73     return current;
74 }
75 }
```

2.

```
// 2. method cariMinIPK dan cariMaxIPK
public void cariMinIPK() {
    if (isEmpty()) {
        System.out.println(x: "Binary tree kosong");
        return;
    }
    Node18 current = root;
    while (current.left != null) {
        current = current.left;
    }
    System.out.println(x: "Mahasiswa dengan IPK terkecil:");
    current.mahasiswa.tampilInformasi();
}

public void cariMaxIPK() {
    if (isEmpty()) {
        System.out.println(x: "Binary tree kosong");
        return;
    }
    Node18 current = root;
    while (current.right != null) {
        current = current.right;
    }
    System.out.println(x: "Mahasiswa dengan IPK terbesar:");
    current.mahasiswa.tampilInformasi();
}
```

3.

```
214 // 3. method tampilMahasiswaIPKdiAtas
215 public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
216     System.out.println("Data mahasiswa dengan IPK di atas " + ipkBatas + ":");
217     tampilMahasiswaIPKdiAtas(root, ipkBatas);
218 }
219
220 private void tampilMahasiswaIPKdiAtas(Node18 node, double ipkBatas) {
221     if (node != null) {
222         tampilMahasiswaIPKdiAtas(node.left, ipkBatas);
223         if (node.mahasiswa.ipk > ipkBatas) {
224             node.mahasiswa.tampilInformasi();
225         }
226         tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
227     }
228 }
```

4.

```
26 public void add(Mahasiswa18 data) { You, 6 minutes ago • Uncommitted changes
27     if (dataMahasiswa[0] == null) {
28         dataMahasiswa[0] = data;
29         idxLast = 0;
30         return;
31     }
32
33     int currentIndex = 0;
34     while (true) {
35         if (data.ipk < dataMahasiswa[currentIndex].ipk) {
36             int leftChild = 2 * currentIndex + 1;
37             if (leftChild >= dataMahasiswa.length) {
38                 expandArray(leftChild);
39             }
40             if (dataMahasiswa[leftChild] == null) {
41                 dataMahasiswa[leftChild] = data;
42                 if (leftChild > idxLast) {
43                     idxLast = leftChild;
44                 }
45                 break;
46             }
47             currentIndex = leftChild;
48         } else {
49             int rightChild = 2 * currentIndex + 2;
50             if (rightChild >= dataMahasiswa.length) {
51                 expandArray(rightChild);
52             }
53             if (dataMahasiswa[rightChild] == null) {
54                 dataMahasiswa[rightChild] = data;
55                 if (rightChild > idxLast) {
56                     idxLast = rightChild;
57                 }
58                 break;
59             }
60             currentIndex = rightChild;
61         }
62     }
63 }
64
```

5.

```
64
65 private void expandArray(int minIndex) {
66     int newSize = Math.max(dataMahasiswa.length * 2, minIndex + 1);
67     Mahasiswa18[] newArray = new Mahasiswa18[newSize];
68     System.arraycopy(dataMahasiswa, srcPos: 0, newArray, destPos: 0, dataMahasiswa.length);
69     dataMahasiswa = newArray;
70 }
71
72 public void traversePreOrder() {
73     traversePreOrder(idxStart: 0);
74 }
75
76 public void traversePreOrder(int idxStart) {
77     if (idxStart <= idxLast && idxStart < dataMahasiswa.length) {
78         if (dataMahasiswa[idxStart] != null) {
79             dataMahasiswa[idxStart].tampilInformasi();
80             traversePreOrder(2 * idxStart + 1);
81             traversePreOrder(2 * idxStart + 2);
82         }
83     }
84 }
85 }
86
```